



UNIVERSIDADE FEDERAL DE ITAJUBÁ

# **Algoritmos e Estrutura de Dados I**

**CTC001**

**Árvore de Expressão Aritmética**

**Vanessa Cristina Oliveira de Souza**



# Introdução

---

- ▶ A árvore binária pode ter diversas variações
  - ▶ Árvore binária de busca
    - ▶ Balanceadas
      - AVL
      - Rubro-Negra
  - ▶ Árvores Heap
  - ▶ Árvores de Decisão
  - ▶ Árvores binárias de expressão
    - ▶ Árvores de expressão aritmética





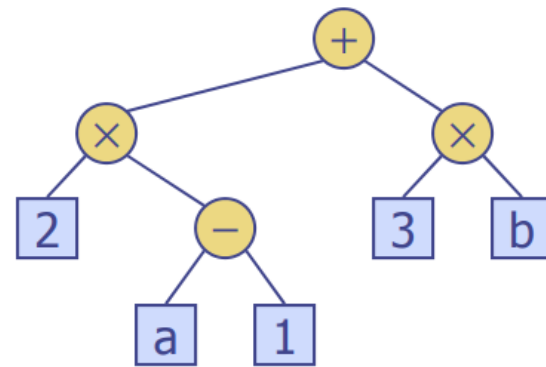
# Introdução

---

- ▶ O conceito principal de uma Árvore Binária de Expressão Aritmética é representar expressões matemáticas de forma hierárquica, onde cada operação é um nó interno e cada operando é um nó folha.

- ▶ Exemplo:

- ▶  $(2 * (a - 1) + (3 * b))$





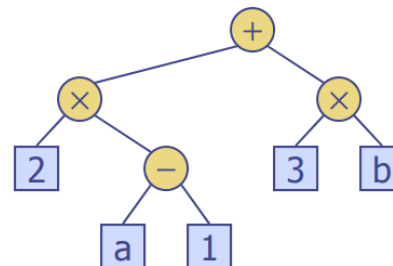
# Introdução

---

- ▶ Quando precisamos computar uma expressão como essa, o compilador **NÃO** avalia diretamente.
- ▶ Ele transforma o código em **árvores de expressão** (ASTs).
- ▶ É assim que o computador descobre:
  - ▶ o que deve ser feito primeiro,
  - ▶ como associar operações,
  - ▶ como gerar bytecode/máquina.
- ▶ **Sem árvore de expressão, uma linguagem de programação não existe.**

- ▶ **Exemplo:**

- ▶  $(2 * (a - 1) + (3 * b))$





# Introdução

---

- ▶ O mesmo vale para consultas em bancos de dados
  - ▶ `SELECT nome FROM alunos WHERE idade > 18 AND curso = "SI"`

```
      AND
    /   \
idade > 18 curso = 'SI'
```





# Introdução

---

- ▶ Uma árvore de expressão é a forma como o computador entende matemática.
- ▶ Toda vez que você escreve código, uma fórmula no Excel, uma consulta SQL ou um shader GPU, o computador transforma isso em árvores, porque árvores deixam claro o que deve ser feito, em qual ordem, e como combinar os valores.
- ▶ Sem árvores de expressão, não existiriam linguagens de programação, bancos de dados nem jogos modernos.



# Árvore de Expressão Aritmética



# Árvore de Expressão Aritmética

---

- ▶ Tecnicamente, a árvore de expressão aritmética é uma ***Arithmetic Expression Tree***, que é um caso específico de ***Expression Tree*** e também um subtipo de ***Abstract Syntax Tree (AST)*** usado para representar operações matemáticas.







# Árvore de Expressão Aritmética

---

- ▶ Para gerar uma árvore de expressão aritmética existem algumas abordagens diferentes.
- ▶ Neste curso, adotaremos o seguinte processo:
  - ▶ A partir da expressão aritmética infixa, primeiro convertemos para expressão pós-fixada (também chamada de notação polonesa reversa), utilizando o algoritmo *Shunting Yard Algorithm*.
  - ▶ A partir da expressão pós-fixada, construímos a árvore usando o algoritmo de construção de árvore via pilha (*Stack-Based Expression Tree Construction*).
  - ▶ Com a árvore pronta, avaliamos o resultado da expressão realizando um percorrimento em pós-ordem (esq-dir-raiz), que corresponde exatamente à lógica da pós-fixada.





## *Shunting Yard Algorithm*

---

- ▶ Desenvolvido por Edsger Dijkstra para converter expressões matemáticas da notação infixa para a notação posfixa, também conhecida como Notação Polonesa Reversa.
- ▶  $(2 * (a - 1) + (3 * b))$
- ▶  $2\ a\ 1\ -\ *\ 3\ b\ *\ +\ *$





## Shunting Yard Algorithm

---

- ▶ O algoritmo Shunting-Yard utiliza duas estruturas de dados principais:
- ▶ Uma fila de saída (*output queue*) para armazenar a expressão na notação posfixa.
- ▶ Uma pilha de operadores (*operator stack*) para armazenar temporariamente os operadores e parênteses.
- ▶ O algoritmo processa a expressão infixa, um token de cada vez, da esquerda para a direita, seguindo um conjunto de regras baseadas na precedência e associatividade dos operadores.





# *Shunting Yard Algorithm*

---

- ▶ **Regras**
- ▶ Se o token for um número (operando)
  - ▶ Adicione-o diretamente à fila de saída.
- ▶ Se o token for um operador
  - ▶ Enquanto houver um operador no topo da pilha com maior ou igual precedência (e for associativo à esquerda), retire esse operador da pilha e adicione-o à fila de saída.
  - ▶ Empilhe o operador atual.
- ▶ Se o token for um parêntese de abertura (
  - ▶ Empilhe.
- ▶ Se o token for um parêntese de fechamento )
  - ▶ Retire os operadores da pilha e adicione-os à fila de saída até encontrar o parêntese de abertura
  - ▶ Descarte o parêntese de abertura (não o adicione à fila de saída).
- ▶ Após processar todos os tokens
  - ▶ Retire quaisquer operadores remanescentes da pilha e adicione-os à fila de saída.





## *Shunting Yard Algorithm*

---

- ▶ **Exemplo**

- ▶  $(2 * (a - 1) + (3 * b))$

- ▶  $2\ a\ 1\ -\ *\ 3\ b\ *\ +\ *$





## Árvore de Expressão Baseado em Pilha

---

- ▶ O algoritmo de construção de Árvore de Expressão Baseado em Pilha (*Stack-Based Expression Tree Construction*) é um método eficiente para criar uma representação em árvore binária de uma expressão matemática na notação posfixa (ou RPN).
- ▶ O algoritmo utiliza uma única pilha para armazenar ponteiros para subárvores à medida que são construídas.





# Árvore de Expressão Baseado em Pilha

---

## ▶ Algoritmo

### ▶ Inicializar uma Pilha

- ▶ Crie uma pilha vazia que armazenará os nós (ou subárvores) da árvore.

### ▶ Ler a Expressão Posfixa

- ▶ Percorra a expressão posfixa, um símbolo (token) de cada vez, da esquerda para a direita.

### ▶ Processar Operando:

- ▶ Se o símbolo lido for um operando (um número ou variável), crie um novo nó de árvore (que é, inicialmente, uma árvore de um único nó ou folha) contendo esse operando.
- ▶ Empilhe este novo nó na pilha.

### ▶ Processar Operador:

- ▶ Se o símbolo lido for um operador (como +, -, \*, /), crie um novo nó de árvore contendo esse operador como a raiz.
- ▶ Desempilhe os dois nós do topo da pilha:
- ▶ O primeiro nó desempilhado torna-se o filho direito do novo nó operador.
- ▶ O segundo nó desempilhado torna-se o filho esquerdo do novo nó operador.
- ▶ Empilhe um ponteiro para este novo nó operador (que agora é a raiz de uma subárvore) de volta na pilha.

### ▶ Concluir:

- ▶ Continue os passos 3 e 4 até que todos os símbolos da expressão posfixa tenham sido processados.
- ▶ Ao final, a pilha conterá exatamente um único ponteiro para o nó raiz da árvore de expressão completa.
- ▶ Desempilhe este último nó; ele é a raiz da sua árvore de expressão final





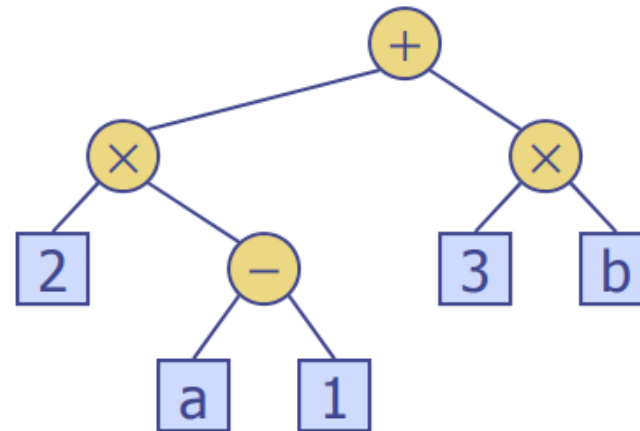
# Árvore de Expressão Baseado em Pilha

---

## ▶ Exemplo

▶  $(2 * (a - 1) + (3 * b))$

▶  $2 \ a \ 1 \ - \ * \ 3 \ b \ * \ + \ *$







# Pseudocódigo

---

**Algorithm 1** Shunting Yard (conversão infix a  $\rightarrow$  pós-fixada)

---

**Require:** expressão infix a tokenizada

**Ensure:** fila output contendo a expressão pós-fixada

```
1: Crie fila vazia output
2: Crie pilha vazia op
3: for cada token  $t$  na expressão do
4:   if  $t$  é um operando then
5:     Enfileire  $t$  em output
6:   else if  $t$  é um operador  $o_1$  then
7:     while op não vazia e topo(op) é operador  $o_2$  e  $(\text{precedence}(o_2) >$ 
      precedence( $o_1$ )  $\vee (\text{precedence}(o_2) = \text{precedence}(o_1)$  do
8:        $o_2 \leftarrow$  desempilhe(op)
9:       Enfileire  $o_2$  em output
10:    end while
11:    Empilhe  $o_1$  em op
12:   else if  $t$  é ( then
13:     Empilhe  $t$  em op
14:   else if  $t$  é ) then
15:     while topo(op)  $\neq$  ( do
16:        $o \leftarrow$  desempilhe(op)
17:       Enfileire  $o$  em output
18:     end while
19:     Desempilhe ( de op           $\triangleright$  descarta o parêntese esquerdo
20:   end if
21: end for
22: while op não vazia do
23:    $o \leftarrow$  desempilhe(op)
24:   Enfileire  $o$  em output
25: end while
26: return output
```

---



# Pseudocódigo

## Algorithm 2 Construção de Árvore de Expressão via Pilha (pós-fixada)

**Require:** lista de tokens em notação pós-fixada

**Ensure:** ponteiro para a raiz da árvore de expressão

```
1: Crie uma pilha vazia  $S$ 
2: for cada token  $t$  na expressão pós-fixada do
3:   if  $t$  é um operando then
4:     Crie um nó  $N$  com valor  $t$  e filhos nulos
5:     Empilhe  $N$  em  $S$ 
6:   else ▷  $t$  é um operador
7:      $R \leftarrow$  desempilhe( $S$ ) ▷ filho direito
8:      $L \leftarrow$  desempilhe( $S$ ) ▷ filho esquerdo
9:     Crie um nó  $N$  com valor  $t$ , filho esquerdo  $L$  e direito  $R$ 
10:    Empilhe  $N$  em  $S$ 
11:   end if
12: end for
13: return topo( $S$ ) ▷ único nó restante é a raiz
```



# Pseudocódigo

## Algorithm 3 Avaliação de uma Árvore de Expressão (pós-ordem)

**Require:** ponteiro para o nó raiz

**Ensure:** valor numérico da expressão

```
1: function AVALIAR(no)
2:   if no == NULL then
3:     return 0
4:   end if
5:   if no é um operando then
6:     return valor(no)
7:   end if
8:    $L \leftarrow \text{AVALIAR}(\text{no.esq})$ 
9:    $R \leftarrow \text{AVALIAR}(\text{no.dir})$ 
10:  if no.operador == '+' then
11:    return  $L + R$ 
12:  else if no.operador == '-' then
13:    return  $L - R$ 
14:  else if no.operador == '*' then
15:    return  $L \times R$ 
16:  else if no.operador == '/' then
17:    return  $L \div R$ 
18:  else if no.operador == '^' then
19:    return thenPow(L, R)
20:  else
21:    return 0                                ▷ operador desconhecido
22:  end if
23:
```



## Exercícios

---

- ▶ Desenhe as árvores que correspondem às seguintes expressões aritméticas:

a)  $2 * (a - b / c)$

b)  $a + b + 5 * c$





## Exercícios

- A partir da estrutura de dados do tipo "Árvore", mostrada na Figura 3, é possível construir uma expressão aritmética, que, ao ser resolvida, resultará em um número situado dentro do seguinte intervalo:

- a)  $-9 \leq \text{resultado} \leq -5$ .
- b)  $-4 \leq \text{resultado} \leq 0$ .
- c)  $1 \leq \text{resultado} \leq 5$ .
- d)  $6 \leq \text{resultado} \leq 10$ .
- e)  $11 \leq \text{resultado} \leq 15$ .

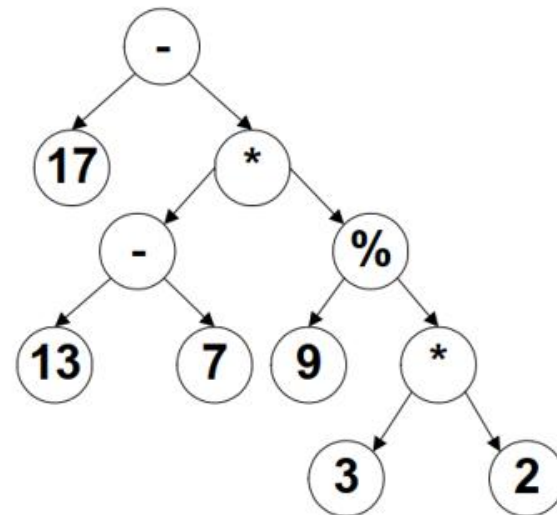


Figura 3 - Estrutura de dados, do tipo "Árvore"